Running the specs now triggers an error because Rails expects to find a template file at /app/views/projects/new.html.erb. After you create a blank file in that spot, you see a Capybara error:

```
1) adding a project allows a user to create a project with tasks
   Failure/Error: fill_in "Name", with: "Project Runway"

   Capybara::ElementNotFound:
     Unable to find field "Name"
```

Capybara searches for form items by DOM ID, form name, or the text of the associated label, whichever it finds first. You're presumably using the label, but since the view file is blank, it isn't there. You have three form elements to take care of: a text field for the name, a multiline text area for the tasks, and a submit button.

With the understanding that in a real project you would care about things like "design" and "making it not look ugly," just put in a basic form that matches your current needs:

test_first/01/app/views/projects/new.html.erb

```
<h1>New Project</h1>

<%= form_for(@project) do |f| %>
  <%= f.label(:name) %>
  <%= f.text_field(:name) %>
  <br />
  <%= f.label(:tasks) %>
  <%= text_area_tag("project[tasks]") %>
  <br />
  <%= f.submit %>
<% end %>
```

This is boilerplate, with one exception—you're creating the text area for tasks by using a text_area_tag rather than the ActiveRecord data-aware text_area method. This is because you're going to do some processing on the list of tasks, and tasks isn't a basic attribute of Project. If you do use f.text_area :tasks, Rails tries to make the value of the text area the value of the tasks relation, and places in that text area an ugly Ruby string representation of the empty relation, which is not what you want.

At this point the test will submit the form and fail. Now the test is looking for the create method in the controller that is invoked by submitting the form.

Going with the Workflow

Now you need to make some decisions. You have some logic that goes beyond Rails boilerplate—namely, you need to parse that list of tasks and create Task